PROG1384

Développement Web côté serveur



ccnb.ca

Passer des données via l'URL

PHP et `$_GET`

Objectif : comprendre comment envoyer des informations à une page PHP sans nécessairement utiliser un formulaire.

Ce que l'on connaît déjà

Avec un formulaire en méthode `GET`, les données sont visibles dans l'URL.

```
<form action="afficherPaysTextBox.php" method="get">
  <input type="text" name="txtPays" />
  <button type="submit">Rechercher</button>
</form>
```

Exemple d'URL générée :

```
http://localhost/php/afficherPaysTextBox.php?txtPays=Hi
```

Ce que le formulaire fait pour nous

Lorsque l'utilisateur clique sur le bouton :

1. le navigateur lit les champs du formulaire;
2. il prend les valeurs des attributs `name` ;
3. il ajoute les données à la fin de l'URL;
4. la page PHP reçoit les données dans `$_GET` .

```
txtPays=Hi
```

`txtPays` est le nom du champ.

`Hi` est la valeur entrée par l'utilisateur.

Construire l'URL nous-mêmes

On peut aussi passer des données sans formulaire.

Il suffit de construire une URL avec des paramètres.

```
bonjour.php?nom=Dupont&prenom=Jean
```

La page `bonjour.php` recevra :

```
$_GET["nom"]
$_GET["prenom"]
```

Anatomie d'une URL avec paramètres

```
page.php?param1=valeur1&param2=valeur2
```

- `page.php` : la page appelée
- `?` : début des paramètres
- `param1=valeur1` : une paire nom / valeur
- `&` : séparateur entre les paramètres

Chaque paramètre doit avoir un nom clair.

Exemple simple

Lien vers une page `bonjour.php` :

```
<a href="bonjour.php?nom=Dupont&amp;prenom=Jean"> Dis-moi bonjour ! </a>
```

Important en HTML :

```
&amp;
```

On écrit `&` dans le code HTML, même si l'URL affiche `&`.

Récupérer les données en PHP

Dans `bonjour.php` :

```php
$nom = $_GET["nom"];
$prenom = $_GET["prenom"];

$message = "Bonjour " . $prenom . " " . $nom;
```

Mais attention : cette version suppose que les paramètres existent.

Il faut éviter de faire confiance directement à l'URL.

Vérifier si le paramètre existe

Avant d'utiliser une valeur de `$_GET`, on vérifie qu'elle existe.

```php
if (isset($_GET["nom"])) {
    $nom = $_GET["nom"];
} else {
    $nom = "visiteur";
}
```

Ne jamais faire confiance à l'URL

L'utilisateur peut modifier l'URL manuellement.

```
bonjour.php?nom=<script>alert('test')</script>
```

Les données provenant de `$_GET` doivent être considérées comme externes et non fiables.

On doit les vérifier et les protéger avant de les afficher.

Afficher une valeur de façon sécuritaire

Lorsqu'on affiche une valeur venant de `$_GET`, on utilise `htmlspecialchars()`.

```php
$nom = htmlspecialchars($_GET["nom"]);

echo "Bonjour " . $nom;
```

Cela aide à éviter l'exécution de code HTML ou JavaScript injecté.

Générer des liens dynamiques

Exemple : créer un lien vers la fiche d'un pays.

```php
$id = 5;
$nom = "Canada";

$url = "detailsPays.php?id=" . $id;

echo '<a href="' . $url . '">' . $nom . '</a>';
```

Le lien généré :

```
detailsPays.php?id=5
```

Construire une URL plus proprement

Pour plusieurs paramètres, `http_build_query()` est pratique.

```php
$params = [
    "nom" => "Dupont",
    "prenom" => "Jean"
];

$url = "bonjour.php?" . http_build_query($params);
```

Résultat :

```
bonjour.php?nom=Dupont&prenom=Jean
```

Exemples réels

Plusieurs sites passent des informations dans l'URL.

```
youtube.com/watch?v=IeJ5UMcoh_8
```

```
google.com/maps?q=45.5017,-73.5673
```

```
quizizz.com/join?gc=507548
```

Les paramètres permettent d'indiquer quoi afficher ou quoi chercher.

À retenir

- `GET` envoie les données dans l'URL.
- `?` commence la liste des paramètres.
- `&` sépare les paramètres.
- PHP récupère les valeurs avec `$_GET`.
- On vérifie toujours si les valeurs existent.
- On protège toujours l'affichage avec `htmlspecialchars()`.
- On utilise des requêtes préparées pour les données SQL.